

# Fast-Forwarding LSM Tree Dataset Loading for Fast and Realistic Benchmarking

## Abstract

Benchmarking key-value (KV) stores is an essential step in storage system development, performed repeatedly to evaluate performance and reliability. Each evaluation should first construct a test dataset, which developers typically build by sequentially inserting a synthetic KV sequence. While this natural accumulation produces a realistic testbed, it becomes a severe bottleneck at scale. On log-structured merge-tree (LSM-Tree) based KV stores, compaction repeatedly rewrites the same data, and the resulting write amplification makes loading a multi-terabyte dataset prohibitively slow.

We introduce *F2Load*, which fast-forwards natural accumulation while removing its main cost: accessing the actual key-value data. Our key insight is that the cumulative outcome of compaction can be predicted from only a compact summary of each Sorted String Table (SST). *F2Load* replaces each SST with an in-memory virtual SST (vSST) and runs virtual compaction along the original compaction path, materializing real SSTs only once the final LSM-Tree state is fixed. This simulate-in-memory, write-once loading reduces loading time by up to  $47.9\times$  over natural accumulation, while reproducing its LSM-Tree access behavior and performance on YCSB and MixGraph. In particular, it builds an 8 TB dataset with 91 B KV size in under two hours, a load the baseline would need a projected 47 hours to finish.

## 1 Introduction

Performance evaluation in storage system development is not a one-time task. To validate a new design, configuration, hardware, or workload, developers build an evaluation environment, run a workload, analyze the results, and repeat with different parameters [12, 26]. For the results to reflect a real system, the environment must reproduce the scale and workload characteristics of production as closely as possible. However, using production data directly is hard. Not only are production traces costly to collect and a privacy risk, but they also fix the scale, key distribution, and request

mix, so they cannot cover varied experimental conditions. Performance evaluation in systems research therefore relies on synthetic workloads and synthetic datasets, such as YCSB [9], LinkBench [2], TAOBench [8], and MixGraph [6].

Among synthetic storage workloads, we focus on dataset loading for log-structured merge-tree (LSM-Tree) based key-value (KV) stores. These stores are actively studied, enhanced [18, 20, 35, 38], and widely deployed in production [11, 21, 34]. Various storage applications also adopt them as a storage backend [ ] and various performance evaluations of storage components, such as databases, file systems, and storage devices, also uses them as benchmarking workloads [4, 13, 17, 25]. **JK: 이런 말을 하고싶었던건가?** Since every such evaluation requires a dataset to run against, dataset loading exists as a first step in the evaluation process.

To load a dataset, developers usually run a synthetic write-only workload against the store. The LSM-tree inserts incoming KV pairs into an in-memory buffer called a memtable. When the memtable fills, the LSM-tree flushes its contents to disk as a Sorted String Table (SST). Compaction then merges and rewrites the resulting SSTs, progressively building the dataset across the LSM-tree’s levels.

The problem is that this incremental construction—loading the dataset progressively through compaction—becomes expensive at large scale. Modern services generate increasing amounts of data [28]. Facebook reports hundreds of petabytes across more than 30 applications [11]. Realistic evaluation therefore needs synthetic datasets at the terabyte scale as well. At that scale, however, incremental construction can take tens of hours and can become a bottleneck in the evaluation process. At the 91 B KV size that Facebook reports for its production workload [6], loading an 8 TB dataset takes 46.2 hours.

The cause is the write amplification of incremental construction, which rewrites the same data many times through compaction. As new data is loaded, compaction merges it with overlapping SSTs already stored in the LSM-tree. In the 8 TB experiment, compaction wrote over 160 TB of SST data. Each rewrite also consumed CPU time to merge keys and

rebuild SST filters and indexes. The loading process therefore wrote tens of times the final dataset’s size, while repeatedly paying the CPU cost of SST construction.

In the specific context of dataset loading, a subsequent workload accesses only the SSTs in the final LSM-tree state. In the 8 TB load, only 3.59% of all SSTs written during loading remain in the final LSM-tree state. The rest are intermediate SSTs that are replaced during loading. Therefore, a loader that directly builds the final LSM-tree state without materializing intermediate SSTs could reduce the I/O and CPU costs of dataset loading.

Skipping the materialization of intermediate SSTs is promising, but the loader must reproduce the final LSM-tree state that incremental construction produces. This state includes the DB size, the number of populated levels, and the placement and overlap of SSTs. It determines which SSTs a lookup accesses and how much data subsequent compactions rewrite, thereby shaping subsequent workload behavior. If the loader does not accurately reproduce this state, the resulting dataset can yield misleading evaluation results, regardless of loading speed.

The final LSM-tree state also changes with the loading configuration. Changes in dataset size, key uniqueness, and SST options affect the resulting DB size and SST placement across levels. A loader that builds the final LSM-tree state directly must therefore meet two goals.

- **Goal 1. Fidelity.** The loader must reproduce the final LSM-tree state that incremental construction produces to preserve subsequent workload behavior.
- **Goal 2. Flexibility.** The loader must reproduce the corresponding final LSM-tree state across the loading configurations that experiments require.

To meet these goals, we propose F2Load, which simulates incremental construction in memory and writes SSTs only once. F2Load represents each SST as an in-memory descriptor called a *virtual SST* (*vSST*). Each *vSST* stores SST metadata and compact summaries of its key distribution and key set. Using these descriptors, F2Load performs *virtual compaction* to predict the output SSTs that real compaction would produce and update the LSM-tree state in memory. F2Load repeats virtual compaction until it reaches the final LSM-tree state. It then materializes the final *vSSTs* as real SSTs in a single materialization phase. This approach reproduces the final LSM-tree state of incremental construction while avoiding the I/O and CPU costs of repeatedly constructing intermediate SSTs.

We implement F2Load on RocksDB and evaluate it on synthetic datasets of up to 8 TB. Compared with incremental construction, F2Load accelerates dataset loading by up to  $47.9\times$  and reduces total disk writes by up to 96.6%. The resulting datasets exhibit similar access behavior, throughput, and latency under YCSB and MixGraph workloads to those loaded through incremental construction. We further evaluate fidelity across different loading configurations and show

that F2Load reproduces the corresponding final LSM-tree states while preserving subsequent workload behavior. Our contributions are summarized as follows:

- We analyze the I/O and CPU costs of incremental construction and identify repeated SST construction as a major source of loading overhead. We show that most SSTs produced during loading are absent from the final LSM-tree state, motivating dataset loading without materializing intermediate SSTs (Section 2).
- We demonstrate that the final LSM-tree state affects subsequent workload behavior and depends on the loading configuration. These findings establish fidelity and flexibility as requirements for direct construction of the final LSM-tree state (Section 3).
- We introduce virtual SSTs and virtual compaction to predict and apply compaction-induced state changes in memory. This process derives the final LSM-tree state without materializing intermediate SSTs, followed by a single materialization phase for the final SSTs (Section 4).
- We implement F2Load on RocksDB and evaluate its loading performance, fidelity, and flexibility. Our evaluation quantifies reductions in loading time and disk writes and compares the final LSM-tree states and subsequent workload behavior with those of incremental construction across multiple loading configurations (Section 5).

## 2 Background and Motivation

### 2.1 Dataset Loading in LSM-Tree Evaluation

Developers evaluate a storage system many times during its development, not once. To check whether a change improves performance, they set up the evaluation environment, run the workload, analyze the results, and try different parameters. This practice is known as performance regression testing, or continuous benchmarking when it runs as part of the build pipeline [12, 26]. MongoDB’s Distributed Systems Infrastructure, for example, automates the whole cycle [14].

These evaluations require datasets at representative scales and with data characteristics relevant to the target application. Production data and traces are often difficult to obtain because of collection costs and privacy concerns. They also reflect particular operating conditions, limiting the range of experiments they can support. To address these limitations, developers use synthetic datasets and workloads whose scale and characteristics can be configured for each experiment. Benchmarks such as YCSB [9], LinkBench [2], and TAOBench [8] provide this capability.

We focus on synthetic dataset loading for LSM-tree based KV stores. Dataset loading populates the store with the data on which a subsequent evaluation workload operates. YCSB separates this process into a *load* phase that populates the database and a *run* phase that executes the evaluation workload [9, 36]. RocksDB’s *db\_bench* provides write-only work-

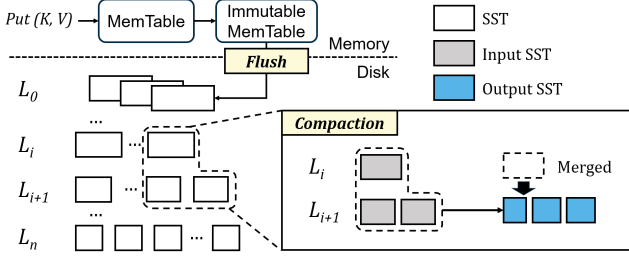


Figure 1: Write path of incremental construction.

loads such as *fillrandom* and *fillseq* for dataset loading [29].

Dataset loading for LSM-tree based KV stores is also required when evaluating other storage components. These evaluations use KV stores as storage backends or application-level benchmarks, so they must populate the stores before executing their workloads. Examples include evaluations of distributed databases, file systems, and storage devices using RocksDB or LevelDB [4, 13, 17, 25].

LSM-tree based KV stores load a dataset through repeated flushes and compactions, a process we call *incremental construction*. Figure 1 illustrates this process. Incoming KV pairs are first inserted into an in-memory buffer called a memtable. When the memtable fills, it becomes immutable and is flushed to disk as a Sorted String Table (SST) in  $L_0$ . In leveled compaction, SSTs selected from  $L_i$  are merged with overlapping SSTs in  $L_{i+1}$ , and the output SSTs are written to  $L_{i+1}$ . The merge discards obsolete versions of keys when they are no longer needed. Repeated flushes and compactions progressively populate the LSM-tree’s levels as loading proceeds.

## 2.2 The Cost of Loading at Scale

LSM-Tree based KV stores are widely adopted as a storage backend for write-heavy workloads and large-scale KV data [5, 7, 16, 27], and the data they hold keeps growing [28]. RocksDB is a representative example, and Facebook reports that it stores hundreds of petabytes of production data across more than 30 applications [11].

Realistic evaluation therefore requires terabyte-scale datasets, but building one is slow. In our experiments on RocksDB, an 8 TB dataset of 1 KB KV pairs takes 13.3 hours to build, and a 16 TB or 32 TB dataset would take roughly 30 or 64 hours (Figure 2a). Moreover, this cost is paid again every time the system is measured, so loading becomes the bottleneck of the whole evaluation.

The long loading time stems from write amplification, a problem inherent to LSM-Trees. As Figure 1 shows, an LSM-Tree buffers incoming KV pairs in a memtable and, once that memtable is full, marks it immutable and flushes it to disk as a Sorted String Table (SST). In the background, compaction merges SSTs to remove duplicate keys, reclaim space, and reduce point-lookup cost. To do so, it reads the overlapping

SSTs of  $L_i$  and  $L_{i+1}$  and writes their merged output back to  $L_{i+1}$ . As loading proceeds, repeated compactions move data from  $L_0$  down to the deeper levels, and the amplification arises from rewriting the same KV pair at each one. We call this loading process *incremental construction*, since flush and compaction build the final state step by step.

As the data volume grows, so does the cost of incremental construction. A larger dataset forms more levels, so each KV pair is rewritten more times. Compaction writes 12.2 times the ingested data at 500 GB and 20.4 times at 8 TB. Loading time therefore rises faster than the dataset itself, by about  $2.2\times$  each time the DB size doubles (Figure 2a). That cost is not only disk traffic. Every rewrite must also merge the input keys and rebuild the table, including its filters and indexes, so each pass consumes CPU as well as I/O. Figure 2c separates these components, and the merge and SST build time is larger than the I/O time in every configuration.

Beyond scale, the evaluation configuration itself can increase loading time. Compression is one example. KV stores commonly enable it to reduce disk usage and I/O [1, 31]. Because it makes each SST smaller, it reduces the number of compactions. This lowers the cumulative build, merge, and I/O time (Figure 2c). However, each compaction must now decompress and recompress its inputs. This added CPU cost outweighs the savings, so the overall loading time still increases (Figure 2b). The KV size has a similar effect. Facebook reports that the average key and value sizes in its production RocksDB workload are 47.9 B and 42.9 B, totaling 91 B per pair [6]. As the uncompressed bars in Figure 2b show, at the same dataset size the 91 B configuration takes about  $3\times$  longer to load than a 1 KB one. At 8 TB, this means 46.2 hours rather than 13.3. The reason is that smaller pairs mean more records, which raises the cost of building filters and indexes during SST construction and increases the number of keys each merge must process.

## 2.3 Opportunity in Dataset Loading

The write amplification that makes loading slow is not spent on the data that a benchmark will later use. Compaction spends it rewriting SSTs that a subsequent compaction soon replaces. In a 1 TB load, only 6.53% of the SSTs ever written survive into the final LSM-Tree state. The rest are built, merged, and deleted during loading.

This cost can be skipped because dataset loading is a special case, not normal service. A KV store in service must answer reads and scans at all times, so every state it passes through is observable and must be correct and queryable. Dataset loading has no such requirement. It is a preparation phase that runs before measurement begins, and no query is ever issued against the states it passes through.

A loader may therefore jump directly to the final state. If it materializes only the SSTs that the final state holds, each KV pair reaches the disk once rather than once for every level it

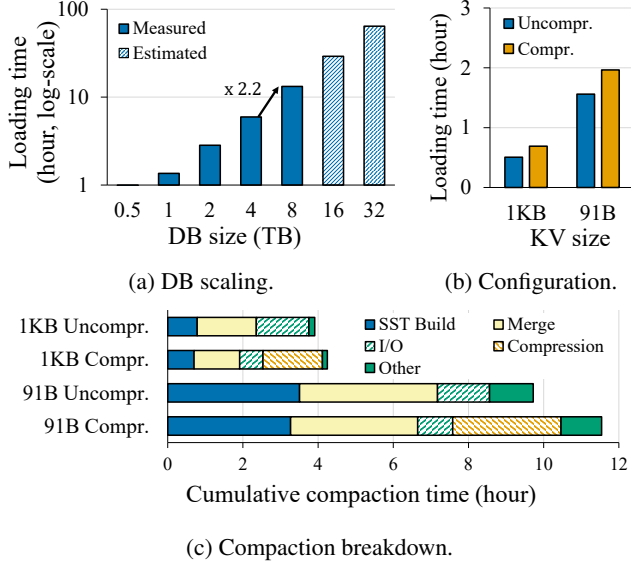


Figure 2: Cost of incremental construction by dataset size and KV size, with and without compression.

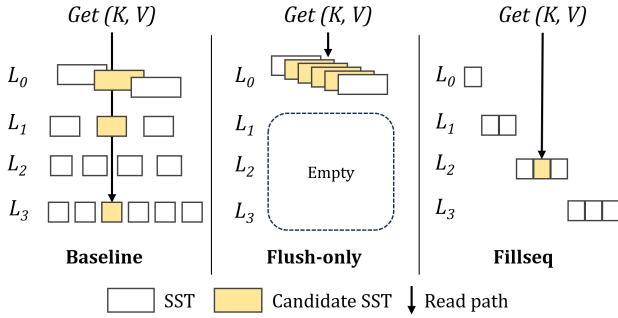


Figure 3: Final state of three loading alternatives, and the SSTs a lookup examines on each.

descends, so the write amplification that dominates loading disappears. What remains is close to the cost of writing the dataset a single time, a small fraction of what incremental construction pays.

### 3 Design Goals for Direct Final-State Construction

Directly building the final LSM-tree state without producing the intermediate SSTs is promising but not straightforward. The difficulty is that the final state is more than a set of SSTs, and that the placement of those SSTs determines the lookup and compaction behavior of a subsequent workload.

A direct final-state construction is therefore useful only if it is fast and also meets two further requirements, *fidelity* and *flexibility*, which the rest of this section makes precise. This section first compares naive approaches to direct final-state construction and shows that the behavior of a subsequent

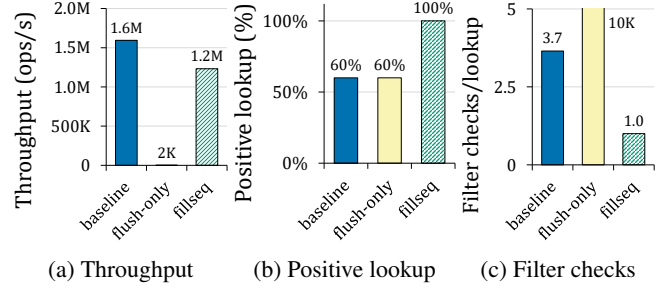


Figure 4: Read behavior of loading alternatives (YCSB C).

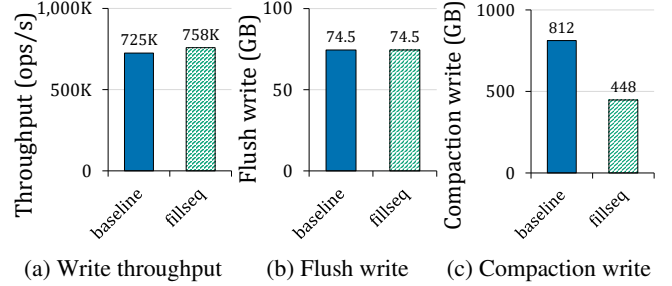


Figure 5: Write behavior of loading alternatives (YCSB A).

workload depends on SST placement. It then shows that the final state a loading configuration produces moves as that configuration changes, which makes that state hard to reuse or to predict.

### 3.1 Effect of SST Placement on Workload Behavior

Compaction 비용을 지불하지 않고 final LSM-tree state에 도달하는 방법은 두 가지를 생각해볼 수 있다. 첫 번째는 로딩 중 compaction을 비활성화하여 flush된 SST를  $L_0$ 에 그대로 누적하는 *flush-only*이고, 두 번째는 key를 정렬된 순서로 삽입하여 새 SST의 key range가 기존 SST와 겹치지 않게 하는 *fillseq*이다 [29]. Key range가 겹치는 SST가 없으면 LSM-tree는 데이터를 다시 쓰는 대신 SST의 level 정보만 변경하는 trivial move를 수행하므로, 후자는 SST를 재작성하지 않고도 여러 레벨에 SST를 배치한다 [?, ?, 30].

두 방법을 1 TB 데이터셋에서 incremental construction과 비교한다. Baseline은 incremental construction으로 24 B key와 1,000 B value로 이루어진 KV pair 약 10억 개를 삽입한다. 각 삽입의 키는 삽입 횟수와 같은 크기의 key space에서 중복을 허용하여 균등하게 추출하므로, 일부 키는 여러 번 기록되고 일부는 한 번도 기록되지 않는다. Flush-only는 동일한 설정에서 compaction만 비활성화하고, fillseq는 key space의 모든 키를 순차적으로 삽입한다.

두 방법은 실제로 로딩을 크게 가속한다. Flush-only와 fillseq는 1 TB 데이터셋을 각각 16.3분과 16.0분에 로딩하여, 59.2분이 걸리는 baseline보다 3.63×와 3.70× 빠르다. Compaction 비용이 없기 때문이다.

그러나 그렇게 만들어진 DB는 baseline의 것과 같지 않

다. Incremental construction의 최종 크기가 767 GiB인 반면 나머지 두 DB는 1 TB에 근접하고, SST 수도 그만큼 많다. Flush-only는 모든 SST를  $L_0$ 에 적재하여 중복 키가 제거되지 않으므로 삽입한 데이터를 그대로 들고 있다. Fillseq는 모든 key를 정렬된 순서로 한 번씩 삽입하여 애초에 중복된 키가 없으므로, key space 전체가 그대로 최종 크기가 된다. SST의 배치도 baseline과 다르다. Flush-only의 SST는 전부  $L_0$ 에 key range가 겹친 상태로 놓이는 반면, fillseq는 모든 레벨에 서로 겹치지 않는 key range의 SST를 배치한다 (Figure 3). 이러한 차이는 후속 워크로드가 실행 중 수행하는 일의 양을 바꾼다.

**Read behavior.** LSM-tree에서 lookup은 key range가 그 key를 포함하는 SST를 후보로 고르고, 후보마다 그 SST의 filter를 확인한다. Filter가 key의 존재를 알리면 해당 SST의 index와 data block을 읽는다. 따라서 lookup 하나의 비용은 filter를 몇 번 확인하고 그중 몇 번이 index와 data block 읽기로 이어지는지로 정해진다.

세 state는 이 두 값에서 갈린다. Figure 4는 세 state에서 YCSB C를 수행한 결과다. Flush-only의  $L_0$  SST들은 서로 key range가 겹치므로 lookup 하나가 거의 모든 SST를 확인하며, 그 수는 lookup당 10,399개에 이르고 throughput은 baseline의 0.1%에 그친다(Figure 4c). Fillseq는 반대로 레벨 사이에 겹침이 없어 filter를 한 번만 확인하지만, 모든 키가 존재하므로 그 한 번이 전부 index와 data block 읽기로 이어진다. Figure 4b의 positive lookup은 lookup 중 실제로 key를 찾아 data block을 읽은 비율이며, fillseq는 100%다. Baseline은 lookup당 filter를 3.65개 확인하고 전체 lookup 중 60.3%만 positive이다. 두 대안은 이 두 값에서 각각 반대 방향으로 벗어난다.

**Write behavior.** 쓰기의 비용은 그 쓰기가 뒤이어 유발하는 compaction 양으로 드러난다. Figure 5는 두 state에 write heavy 워크로드인 YCSB A를 수행하여 데이터셋의 10%에 해당하는 100 GiB를 기록한 결과이다. Compaction write은 워크로드가 끝나는 시점에 아직 수행되지 않은 compaction까지 포함하기 위해 남아 있는 compaction이 모두 끝날 때까지 대기하여 측정했다. Flush-only는  $L_0$ 에 쌓인 compaction debt 때문에 compaction을 치는 순간 전체를 대상으로 하는 단일 compaction이 시작되어 후속 쿼리가 모두 stall되며, 1시간이 지나도 진행되지 않아 측정을 중단했다.

같은 쓰기가 유발하는 compaction 양은 state에 따라 두 배 가까이 갈린다. Baseline과 fillseq는 같은 키를 같은 순서로 기록하므로 flush write은 74.5 GB로 동일하다. Memtable 안에서 중복되는 키는 flush 전에 제거되므로, 워크로드가 기록한 논리적 양인 100 GB보다 적게 측정되었다. 그러나 compaction write은 각 812.0 GB와 447.7 GB로 fillseq가 55%에 그친다(Figure 5). Fillseq는 레벨 사이에 key range 겹침이 적기 때문이다. 데이터가 아래 레벨로 내려갈 때 재작성 대신 trivial move로 처리되거나, 병합하더라도 대상이 적어 compaction 비용이 작다.

이러한 internal behavior의 차이는 throughput을 형성하지만, 드러나는 정도는 실행 환경이 정한다. 위 실험의 50 GiB cache에서는 filter 전체가 cache에 상주하여 확인 비용이

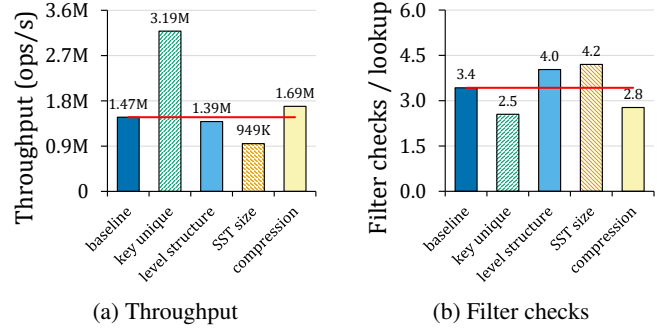


Figure 6: YCSB C on four states, each loaded with one setting changed from the baseline: key uniqueness to 25%, level structure to a 64 MB level base with a multiplier of four, SST size to 256 MB, and compression enabled. The dashed line marks the baseline.

CPU에 머물고 data block 읽기만 device I/O가 된다. 그래서 fillseq는 filter를 적게 확인하고도 baseline보다 느리다. Cache가 작아 filter를 모두 담지 못하는 환경이라면 filter 확인 자체가 device I/O가 되므로, throughput이 반대로 나타날 수 있다. 쓰기 쪽은 드러나는 정도가 더 약하다. Compaction write가 두 배 가까이 다른데도 throughput 차이는 1.05×에 그치는데, 쓰기는 memtable에 기록되고 compaction은 background job이 처리하기 때문이다. 그 차이는 사라진 것이 아니라 background I/O work로 옮겨가 있다.

지금까지의 결과는 SST가 어느 레벨에 놓이고 레벨 사이에 얼마나 겹치는지, 곧 final LSM-tree state가 워크로드의 behavior를 좌우함을 보인다. 그 영향은 간접적으로는 throughput처럼 실행 환경이 함께 섞이는 값에 나타나고, 직접적으로는 query가 수행하는 논리적 동작에 나타난다. 곧 lookup이 확인하는 filter의 수, 그중 key를 찾는 비율, 그리고 쓰기가 유발하는 compaction 양이다. 따라서 direct final-state construction은 빠른 구축만으로 충분하지 않고, 동일한 설정에서 baseline이 만들어내는 final state를 재현해야 한다.

**Goal 1: Fidelity.** 같은 설정에서 incremental construction이 형성하는 final LSM-tree state를 재현해야 한다. State가 다르면 lookup이 확인하는 filter의 수, 그중 key를 찾는 비율, 쓰기가 유발하는 compaction 양이 함께 달라지기 때문이다.

### 3.2 Configuration Dependence of the Final State

Direct final-state construction을 더욱 어렵게 만드는 것은, final state가 로딩 설정에 따라 달라진다는 점이다. Key uniqueness, compression 사용 여부, level structure의 정의, SST size가 달라지면 만들어지는 LSM-tree의 크기와 SST 수, 그리고 SST 사이의 겹침이 함께 달라진다. 앞 절에서 보인 대로 이러한 차이는 후속 워크로드의 behavior로 이어진다.

이를 보여주기 위해 baseline에서 네 가지 설정을 하나씩 바꿔 데이터셋을 로딩하고, 앞 절의 세 지표 중 read behavior를 예로 각 state에 YCSB C를 수행하여 throughput과 filter checks를 측정했다. Figure 6의 baseline은 앞 절에서 밝힌 데이터셋 구성에 더해, level base size 256 MB에 레벨마다 10배씩 늘어나는 구조를 쓰고, SST 하나는 64 MB를 목표로 하며, compression은 사용하지 않는다. 각 범례는 이 baseline에서 설정 하나만 바꾼 로딩을 가리킨다. Key uniqueness를 25%로 낮추거나, level structure를 64 MB level base size에 레벨마다 4배씩으로 바꾸거나, SST size를 256 MB로 키우거나, compression을 사용하도록 한 네 가지다.

이 설정들을 바꾸면 만들어지는 구조는 크게 세 가지 측면에서 달라진다. 첫째는 적재되는 데이터의 총량이다. Key uniqueness는 중복 제거 후 남은 key를 줄이고, compression은 같은 데이터가 차지하는 바이트를 줄인다. 둘째는 레벨 하나가 담는 용량이다. Level structure는 각 레벨의 용량을 바꾸므로, 데이터의 총량이 같아도 쓰는 레벨의 수가 달라진다. 셋째는 그 데이터를 나누는 파일의 크기다. SST size는 레벨별 데이터를 그대로 둔 채 그것이 몇 개의 파일로 나뉘는지를 바꾼다.

그 결과 만들어진 final state는 설정마다 다르다. DB 크기는 0.27 TB에서 0.75 TB까지, SST 수는 3,237개에서 13,539개까지 걸친다. 데이터의 총량이나 레벨 용량을 바꾼 설정은 쓰는 레벨이 달라져서, key uniqueness를 낮춘 state는  $L_4$ 에서 끝나고 level structure를 바꾼 state는  $L_6$ 까지 내려간다. 반면 SST size를 바꾼 state는 레벨별 데이터가 baseline과 같은 채 파일 수만 약 4분의 1로 줄고 평균 SST 크기가 58 MB에서 243 MB가 된다. 어느 설정에서도 로딩은 정상적으로 끝나므로, 이 차이는 오류가 아니라 그 설정이 만드는 state 그 자체다.

Final state의 차이는 workload behavior의 차이로 드러난다. Lookup은 위 레벨부터 후보 SST의 filter를 확인하다가 key를 찾으면 멈추므로, 데이터가 아래 레벨에 몰려 있을수록 확인하는 filter가 늘어난다. Level structure를 바꾼 state는 레벨 하나의 용량이 작아져 데이터의 99%가 가장 아래 레벨에 놓이고, 그만큼 확인하는 filter가 baseline보다 늘어난다. Compression을 켜 state는 레벨 수가 baseline과 같지만, 레벨의 용량이 압축 후 크기로 정해져 위 레벨들이 담는 양은 그대로인 채 가장 아래 레벨만 489 GB에서 113 GB로 줄어든다. 아래 레벨에 놓인 데이터의 비중이 65%에서 30%로 낮아지므로 lookup이 더 위에서 key를 찾고, 확인하는 filter는 줄어든다. Key uniqueness를 낮춘 state는 쓰는 레벨 자체가 하나 줄어 마찬가지로 줄어든다. SST size를 바꾼 state는 레벨 구성이 baseline과 같은데도 SST 하나가 담는 key range가 넓어져 확인하는 filter가 늘어난다. 확인하는 filter가 적은 state일수록 throughput이 높다.

지금까지의 결과는 final state가 로딩 설정에 종속적임을 보인다. 따라서 direct final-state construction은 하나의 설정에서 성립하는 것으로 충분하지 않다. 설정이 바뀌면 재현해야 할 대상 자체가 달라지므로, 설정을 입력으로 받아 그 설정이 만드는 state를 따라갈 수 있어야 한다.

**Goal 2: Flexibility.** 로딩 설정이 달라져도 그 설정의 incremental construction이 형성하는 final state를 재현해야

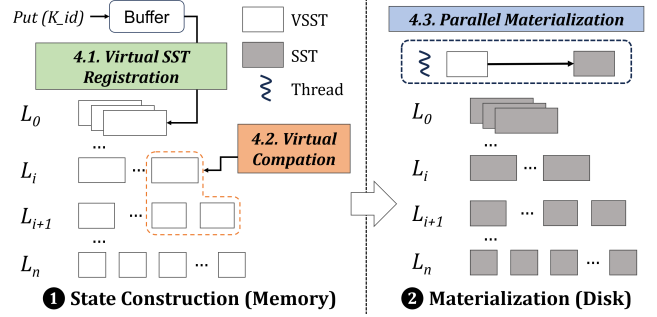


Figure 7: Overview of the dataset loading in F2Load.

한다. 재현 대상 자체가 설정마다 달라지므로, 하나의 설정에서만 성립하는 구축 방법으로는 부족하다.

### 3.3 Reusing or Predicting the Final State

한 번 만든 state를 snapshot이나 백업으로 저장해 두고 재사용하는 방식은 두 goal을 함께 만족하지 못한다. 저장된 state는 그것을 만든 설정에서만 incremental construction의 결과와 일치하므로, 설정이 하나라도 바뀌면 재현해야 할 대상이 달라져 fidelity를 잃는다. 게다가 앞 절의 측들은 서로 독립적으로 조합된다. 네 측에 각각 값을 서너 개씩만 두어도 조합은 수백 개가 되고, DB size와 KV size, compaction policy까지 측으로 더하면 그 수는 곱으로 늘어난다. 이 조합들을 미리 갖추려면 각각에 대해 baseline 로딩을 되풀이해야 하므로, 로딩 비용을 없애는 것이 아니라 앞당겨 치르는 것에 가깝다.

설정으로부터 final state를 계산하거나 예측하는 방식도 성립하기 어렵다. 로딩 설정의 변화는 compaction이 언제 실행되는지, 어떤 SST를 입력으로 고르는지, 그 입력이 어떤 데이터인지, 결과 SST를 어떤 조건으로 끊는지를 바꾼다. 이러한 결정은 compaction job 하나의 동작 단위에서 이루어지며, 그 동작이 반복해서 쌓인 결과가 final state다. 따라서 state의 변화를 따라가지 않고 미리 정의된 수식이나 규칙만으로 final state를 얻기는 어렵다. 결국 incremental construction이 수행하는 compaction을 제거하더라도, final state를 구하려면 그 compaction이 LSM-tree state를 어떻게 바꾸는지 추적해야 한다.

## 4 F2Load Design

In this section, we present our methodology for fast-forwarding dataset loading while achieving fidelity and flexibility. The core principle of F2Load is to follow the incremental construction process while simulating flush and compaction using a small SST descriptor called virtual SST (vSST). All simulation takes place in memory, and F2Load finally materializes the surviving vSSTs into actual SSTs on disk to construct the final state of the LSM-tree. Figure 7 shows F2Load’s loading pipeline.

Table 1: vSST descriptor.

| Component    | Size       | Usage                                        |
|--------------|------------|----------------------------------------------|
| SST Metadata | 40B        | decide compaction trigger and input files    |
| Index models | 32B/model  | approximate key-to-rank function             |
| KMV sketch   | 16B/sample | estimate union cardinality and correct ranks |

- **Virtual SST Registration.** An SST write, whether by a flush or compaction, is replaced by registering a vSST in memory (Section 4.1).
- **Virtual Compaction.** A compaction is replaced by a virtual compaction, which merges input vSSTs and registers the resulting output vSSTs in memory, instead of writing merged SSTs to disk (Section 4.2).
- **Parallel Materialization.** F2Load finally materializes the surviving vSSTs by generating actual SSTs and writing them to disk (Section 4.3).

#### 4.1 Virtual SST Registration

A vSST is a small in-memory structure that retains only the information required for virtual compaction and materialization. It consists of three components as illustrated in Table 1: (1) *SST metadata* to follow the compaction policy, (2) a set of *index models* to approximate the key distribution of the SST, and (3) a *KMV sketch* to deduplicate keys during merging. We describe each component in detail as follows.

**SST Metadata.** SST metadata retains the information required to simulate compaction during incremental construction. It includes the information needed to reproduce compaction triggering and input file selection, generate output vSSTs, and materialize actual SSTs afterward.

In LSM-trees, compaction triggering depends on whether a level’s total size or file count exceeds a threshold, while input file selection depends on the key ranges of SSTs. Neither requires individual KV pairs in an SST. Accordingly, each vSST stores four fields in SST metadata: file size, key range, SST ID, and KV pair count.

File size is used to maintain the total size of each level. Whenever a vSST is added to or removed from a level, F2Load updates the corresponding aggregate statistics. Key range is used for input file selection: F2Load checks whether vSST ranges overlap to expand the inputs of a virtual compaction across adjacent levels. Once the input vSSTs are selected, their SST IDs identify the corresponding vSST objects to merge. The KV pair count is not required for compaction triggering or input file selection. Instead, F2Load uses it to approximate the result of a virtual compaction, which generates the output vSSTs, and to materialize actual SSTs afterward.

A vSST does not store its level or per-level statistics. Instead, LSM-KVS maintains a separate structure (e.g., version set) that tracks the SSTs in each level, their aggregate statistics,

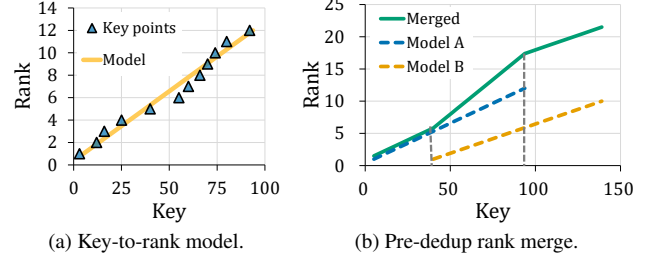


Figure 8: Rank models used by a vSST. (a) A key-to-rank model. (b) The summed rank function before KMV-based deduplication correction.

and SSTs already selected by ongoing compactions. F2Load reuses this structure, replacing SSTs with vSSTs and updating the per-level information as virtual compaction removes input vSSTs and registers output vSSTs.

Because compaction decisions rely only on SST metadata, F2Load does not assume a fixed compaction policy. It can simulate different trigger conditions, input file selection rules, and other policy details without changing the vSST structure.

**Index models.** Index models retain the information required to merge vSSTs without their actual keys. To merge vSSTs, virtual compaction must know the relative position, or rank, of each key in the resulting sorted sequence. This rank information allows F2Load to determine the output vSSTs of a target size, without physically sorting or rewriting any key.

The key-to-rank mapping can be represented exactly only by keeping the

Compaction output을 예측하려면 key space에서 record가 누적되는 위치를 알아야 한다. vSST는 Bourbon과 같이 learned index [10]로 sorted key sequence의 key-to-rank function을 근사한다. Distinct key set  $S = \{k_0 < k_1 < \dots < k_{N-1}\}$ 에 대해 rank는  $k$ 보다 작은 key의 수, 즉  $k$ 가 들어갈 0-based position이다.

$$r_S(k) = |\{x \in S \mid x < k\}|, \quad r_S(k_i) = i.$$

Index는 segment라고도 부르는 여러 piecewise-linear model로 구성되며, 각 model  $j$ 는 key interval  $[l_j, r_j]$ 에서 rank를 예측한다(Figure 8a).

$$r_j(k) = a_j k + b_j, \quad k \in [l_j, r_j].$$

이 방식은 전체 key list 대신 몇 개의 작은 model로 SST의 key distribution을 요약하므로, vSST가 이를 메모리에 유지할 수 있다.

**KMV sketch.** Cross-SST deduplication 결과를 예측하려면 input key set의 union cardinality가 필요하다. 이를 위해 vSST는 K-Minimum Values (KMV) sketch [3]를 유지한다. KMV는 각 distinct key  $k$ 를 deterministic하고 uniform한  $h(k) \in [0, 1)$ 로 mapping하고, hash가 가장 작은  $K$ 개의

$(k, h(k))$  pair만 보관한다. 가장 큰 retained hash가 sampling threshold  $\theta$ 가 된다. 같은 key는 어느 vSST에서도 같은 hash를 가지므로 sample union에서 한 번만 남지만, hash 자체가 모든 duplicate를 식별하는 것은 아니다. Hash는 key set의 균일한 표본을 고르는 역할을 하고, sampled duplicate는 보관된 key identity로 판별한다. 현재 구현은 vSST마다 최대 512개의 global samples를 유지하고, 별도의 512-sample budget을 최대 8개의 range-local sketches에 나누어 사용한다. 이 bounded summary로 전체와 key range별 union cardinality를 추정하는 방법은 Section 4.2에서 설명한다.

초기 vSST는 한 번의 flush가 만들 SST를 in-memory에서 표현한다. Synthetic key generator가 integer key sequence를 생성하면, F2Load은 이를 일반적인 flush 크기의 batch로 나누어 radix-sort한다. 이 시점에는 batch의 actual keys가 있으므로 인접한 duplicate를 정확히 제거한 뒤 PLR model과 KMV sketches를 구축한다(Table 1). 즉 intra-batch deduplication은 exact하고, KMV가 필요한 부분은 이후 여러 vSST 사이의 duplicate이다. 완성된 descriptor는 real SST를 쓰지 않은 채 LSM-tree metadata에 등록되고 virtual compaction의 input이 된다.

## 4.2 Virtual Compaction

일반 compaction은 여러 input을 key order로 merge하고, 같은 user key의 redundant occurrence를 제거한 뒤, deduplicated sequence를 output SST로 split한다. Virtual compaction도 이 dependency를 보존하여 descriptor를 세 단계로 변환한다: (1) input rank function을 합치는 *merge*, (2) KMV estimate로 duplicate rank mass를 제거하는 *deduplication correction*, (3) corrected rank function을 output vSST로 나누는 *split*. 구현에서는 첫 두 단계가 하나의 model-construction pass에 fused되어 있지만, split은 항상 deduplication correction이 끝난 뒤 수행된다.

**Step 1: Merge** F2Load은 Learned-Less Index [15]의 model-merging 기법을 데이터셋 로딩에 적용한다. Input vSST  $i$ 의 distinct key set을  $S_i$ , entry count를  $N_i$ , rank function을  $r_i(k)$ 라 하자. 중복을 아직 고려하지 않은 merged rank는 각 input에서  $k$ 보다 작은 key 수의 합이다.

$$r_{\Sigma}(k) = \sum_i r_i(k), \quad N_{\Sigma} = \sum_i N_i.$$

두 overlapping segment가  $r_A(k) = a_A k + b_A$ 와  $r_B(k) = a_B k + b_B$ 이면  $r_{\Sigma}(k) = (a_A + a_B)k + (b_A + b_B)$ 가 된다(Figure 8b). 이 함수는 input의 전체 rank mass를 보존하므로, 두 SST에 모두 존재하는 key는 두 번 세어진다. 따라서 Figure 8b의 merged line은 deduplication 이전의 intermediate model이다.

**Step 2: Deduplication correction** PLR만으로는  $r_{\Sigma}$ 에서 어느 rank mass가 duplicate인지 알 수 없다. KMV는 모든 duplicate key를 직접 찾아 삭제하는 대신, input union  $\cup_i S_i$ 의 unique cardinality를 추정한다. 각 input sketch의 threshold를  $\theta_i$ 라 할 때, F2Load은 모든 input에서 비교 가능한 공통

sampling probability  $\theta = \min_i \theta_i$ 를 사용한다. 각 sketch에서  $h(k) \leq \theta$ 인 sample만 모으고 같은 key를 한 번만 남긴다. 남은 distinct sample 수를  $m$ 이라 하면, current prototype은 다음 threshold estimator를 사용한다.

$$\hat{U} = \begin{cases} m, & \text{if all sketches are complete,} \\ \min\{N_{\Sigma}, \lceil m/\theta \rceil\}, & \text{otherwise,} \end{cases}$$

$$\hat{D} = N_{\Sigma} - \hat{U}.$$

Complete sketch는 input의 모든 key를 보관하므로 union이 exact하다. 그 외에는 threshold 아래에 union key의  $\theta$  fraction이 표본으로 남는다는 bottom- $K$  sampling 원리로  $\hat{U}$ 를 추정한다 [3]. 여기서  $m/\theta$ , upward rounding, 그리고  $N_{\Sigma}$  cap은 current prototype이 사용하는 구체적인 estimator이다. 예를 들어  $\theta = 0.1$ 에서 두 sketch의 sampled keys가 각각  $\{p, q\}$ 와  $\{q, r\}$ 이면, hash는 세 key를 표본에 포함시키고 key identity 비교가  $q$ 를 한 번만 남겨  $m = 3$ 을 만든다. 따라서 이 구현은 union cardinality를 약 30으로 추정한다.

$\delta = \hat{D}/N_{\Sigma}$ 를 duplicate fraction이라 하면 merged rank에서 남겨야 할 비율은  $\delta$ 가 아니라 unique-retention ratio  $\rho = \hat{U}/N_{\Sigma} = 1 - \delta$ 이다. Duplicate가 key space에 균일하게 분포한다는 단순한 경우에는 사용자가 직관적으로 생각할 수 있는 다음 보정이 성립한다.

$$r_{\text{unique}}(k) \approx \rho r_{\Sigma}(k).$$

그러나 실제 overlap은 key range마다 다를 수 있다. F2Load은 range-local KMV sketches를 이용해 각 PLR breakpoint span  $I_j$ 의 naive rank increment  $M_j = \Delta r_{\Sigma}(I_j)$ 와 unique count  $\hat{U}_j$ 를 구하고,  $M_j > 0$ 인 span에  $\rho_j = \hat{U}_j/M_j$ 를 적용한다. 구현에서 인접 span은 inclusive boundary sample을 공유할 수 있고 각  $\hat{U}_j$ 도 근사치이므로, local estimate의 합을 곧바로 output count로 사용하지 않는다. 대신 normalization factor  $\alpha = \hat{U}/\sum_j \hat{U}_j$ 를 적용하여 corrected rank의 total mass가 global estimate  $\hat{U}$ 와 일치하게 한다.

$$\Delta r_{\text{unique}}(I_j) = \alpha \rho_j \Delta r_{\Sigma}(I_j), \quad r_{\text{unique}}(k_{\max}^+) = \hat{U}.$$

따라서 이 단계의 결과는 특정 duplicate key의 list가 아니라, deduplication 이후의 total entry count와 range별 unique-key density를 표현하는 corrected rank function이다.

**Step 3: Split** F2Load은 deduplication-corrected  $r_{\text{unique}}$ 와  $\hat{U}$ 를 얻은 뒤에만 output을 split한다. Target SST byte size를 average entry size로 나누어 rank-space의 target entry count  $q$ 를 정하고,  $t$ 번째 output boundary를 inverse rank lookup으로 구한다.

$$k_t = r_{\text{unique}}^{-1}(tq).$$

Size boundary뿐 아니라 RocksDB가 lower-level overlap을 제한하기 위해 사용하는 grandparent boundary도 rank space로 변환하여 split decision에 반영한다 [23]. 각 output vSST는 해당 key range의 submodel을 local rank 0으로 rebase하고, 그 range에 속하는 KMV samples를 이어받는다. Split 뒤의 KMV merge는 이미 끝난 deduplication을 다시 수행하는 과정이 아니라, output descriptor가 다음 virtual compaction에서 사용할 sketch를 전파하는 과정이다.

### 4.3 Materialization

더 이상 compaction 작업이 남지 않으면 corrected rank distribution과 output boundary가 모두 확정되어 있다. F2Load은 entry count estimate가  $N_o$ 인 각 final vSST에서 rank  $0, 1, \dots, N_o - 1$ 을 순회하며, PLR의 inverse  $k = r_o^{-1}(p)$ 로 key를 생성해 real SST에 기록을 시도한다. 같은 non-L0 level의 인접 vSST가 boundary key를 공유하면 뒤 range의 lower bound를 앞 range의 maximum보다 1 큰 key로 조정한다. Rounding으로 동일하거나 감소하는 key가 생성되면 이전 key보다 크게 보정하며, 이 보정값이 vSST의 upper bound를 넘으면 해당 range의 생성을 종료한다. 따라서 approximation과 boundary correction 때문에 실제로 기록되는 수는  $N_o$ 보다 작을 수 있다. Deduplication correction은 split 전에 이미 rank count와 distribution에 반영되었으므로, materialization에서 KMV를 다시 적용하거나 final vSST 사이의 key list를 merge하지 않는다. 각 vSST는 독립적으로 materialize할 수 있어 여러 worker가 병렬로 SST를 생성한다. 이후 benchmark workload는 이 real SST들에 RocksDB의 일반 read/write path로 접근한다.

KMV와 PLR은 original key identity 전체를 보관하지 않으므로, 이 과정은 original union을 exact하게 복원하는 것이 아니라 corrected rank마다 하나의 approximate key를 생성한다. 따라서 KMV가 보존하는 것은 deduplicated cardinality와 range별 density이며, current prototype의 live-key membership 한계는 Figure ??에서 별도로 확인한다.

### 4.4 Implementation

F2Load은 Facebook이 개발한 level-based LSM-tree인 RocksDB에 구현하였다. Registered vSST는 RocksDB의 일반 compaction picker에 보이며, picker가 선택한 input과 output level을 그대로 사용해 background thread에서 virtual compaction을 실행한다. 구현은 Step 1의 PLR merge와 Step 2의 range-local KMV correction을 한 번의 model-construction pass로 fuse한 뒤, corrected model을 Step 3의 split routine에 전달한다. Data I/O를 제거한 뒤에는 LSM-tree metadata update가 새로운 병목이 되므로, 여러 VersionEdit을 batch로 처리하고 write-only loading 중 사용하지 않는 metadata work를 미룬다. Materialization이 끝나면 virtual file을 real SST metadata로 교체하여 일반 RocksDB state로 전환한다.

## 5 Evaluation

Experiments use the reference server with Intel Xeon Gold 6336Y CPUs, approximately 1 TiB DRAM and 31 Samsung PM9A3 NVMe SSDs [33] in RAID-0. F2Load is implemented on RocksDB 10.10.1. The common comparisons in Sections ?? and 3 use the explicitly recorded clean RocksDB 11.1.0 release baseline and 16 write buffers; method-specific builds are disclosed there. The historical scaling figure below retains its earlier builds and configurations and must not be read as sharing that baseline. Leveled compaction, kMinOver-

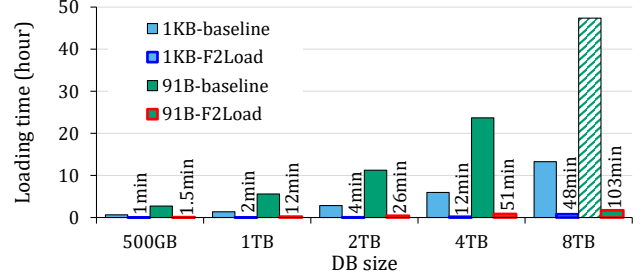


Figure 9: Historical loading-time scaling series, retained separately from the common-release comparisons in Figures ?? and ?. Its original projected points remain hatched. These bars are not a scaling series measured with the new 16-buffer release baseline.

lappingRatio [22], nominal 64 MiB buffers/SST targets, no compression and up to 48 background jobs are common exposed settings, not a claim of identical builds across historical experiments.

Unless stated otherwise, the baseline is db\_bench fillrandom [29], which runs the real RocksDB write and compaction path. The generator writes  $N$  keys in random order, of which about 63.2% are unique. We use two KV sizes, 1 KB (24 B key, 1000 B value) and 91 B (48 B key, 43 B value), and choose  $N$  to reach the target dataset size. For workload runs, we use 48 client threads, run each workload for five minutes, and size the RocksDB block cache [24] to 5% of the dataset.

### 5.1 End-to-End Loading Performance

The historical Figure 9 retains its original size-scaling measurements and projections. Repeating that complete matrix with the common release baseline is deferred. The newly measured common 1,000 GiB, 1 KB comparison instead uses the same input settings and the exact source DBs as Figures ?? and ?.

The separately configured historical series in Figure 9 shows that baseline loading time increases quickly with both DB size and key count, reaching 13.25 hours for the 1 KB dataset and 47 hours for the 91 B dataset at 8 TB. F2Load’s loading time is far shorter, so each bar is annotated with its value. F2Load reaches a speedup of up to 47.9 $\times$ , reducing the loading of a 1 TB dataset of 1 KB pairs from 1.4 hours to 102 seconds. For the 91 B datasets it stays consistent at 25.8–28.0 $\times$  across the measured range, and F2Load builds the projected 8 TB dataset in under two hours.

The new common-release 1,000 GiB conventional results are 3501.4 seconds for 1 KB and 11426.2 seconds for 91 B. In the common 1 KB comparison, F2Load takes 134.7 seconds, including 97.0 seconds for its initial loading process and 37.8 seconds for clean-RocksDB completion to zero pending bytes. The loading-time ratio is 25.99 $\times$ . These values and source

DBs are shared with Figures ?? and ??; they are distinct from the historical scaling series above.

The common 1,000 GiB, 1 KB baseline writes 12.71 times the logical input as flush/compaction SST bytes. Device-level write amplification is a separate metric: 13.50 for Baseline and 0.79 for F2Load, a 94.1% reduction in device writes. The F2Load total includes both materialization and subsequent physical completion compactions; it is not a claim that the measured prototype writes every byte only once.

## 5.2 Comparison against Write-path Optimizations

기존 연구들은 incremental construction의 write path를 유지하면서 background work의 실행 방식이나 compaction 과정에서 이동하는 데이터양을 줄이는 방식으로 write path를 최적화해 왔다. 대표적으로 ADOC은 thread 수와 memtable size를 동적으로 조정하여 flush와 compaction 간의 데이터 흐름을 최적화한다 [37]. 반면 BlobDB는 key와 value를 분리하고 value를 별도의 file에 저장함으로써, compaction 과정에서 발생하는 value 재작성을 줄인다 [19, 32]. Figure ??는 24 B key와 1,000 B value로 구성된 1 TB 데이터셋을 대상으로 Baseline과 두 write-path optimization 기법을 비교한 결과를 보여준다.

**Dataflow tuning.** ADOC은 compaction work를 제거하지 않고 그 input buffer size(memtable, SST)와 compaction을 수행하는 background thread 수를 조절한다. 그러나 본 실험에서 ADOC의 loading time은 68.7분으로, Baseline보다 17.7% 길었다. ADOC은 buffer size를 키움으로써 device-level WAF를 13.50에서 11.16으로 낮추고 compaction 수를 60.1% 줄였지만, compaction당 평균 read size는 257.1 MiB에서 530.5 MiB로, 평균 duration은 0.692초에서 1.328초로 각각 2.06×와 1.92× 증가하였다. Compaction 횟수는 줄었지만 각 job은 더 크고 오래 실행되었다. Compaction의 입력 단위가 커지면 한 번에 L1과 L2로 밀려 내려가는 부하도 그만큼 커진다. RocksDB는 아직 compaction되지 않은 데이터양이 임계값을 넘으면 foreground write를 지연시키거나 정지시키므로, 부하가 이렇게 한꺼번에 쌓이면 임계값을 넘는 일이 더 잦아진다. 그 결과 ADOC의 write stall time은 2,347초에서 3,002초로 27.9% 증가하였고, 이는 감소한 compaction 비용을 상쇄한다.

**Key-value separation.** BlobDB는 key와 value를 분리하여 compaction의 오버헤드를 감소시키기 위한 구조이다. Value를 별도의 파일에 저장하고, SST에는 key와 value의 reference만 저장한다. Compaction은 실제 value 대신 key와 value reference만 처리하므로, 더욱 적은 CPU 비용과 I/O 비용으로 처리될 수 있다. 그 결과 loading time은 28.5분으로 Baseline보다 2.05× 빨라졌다. 이 실험에서는 최대 가속화 정도를 측정하기 위해 value가 저장된 파일을 주기적으로 정리하는 garbage collection (GC)을 비활성화하였다. 실험이 24 B라는 작은 key size를 대상으로 하였기에 로딩 가속화를 더욱 극대화할 수 있었다.

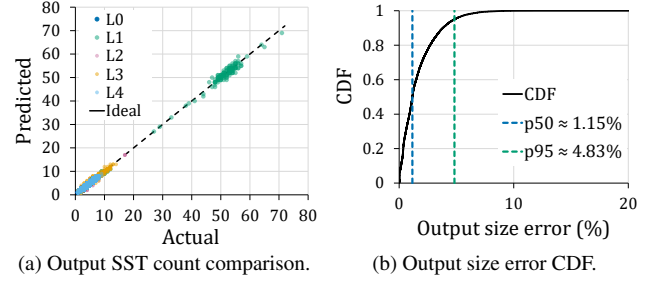


Figure 10: Accuracy of virtual compaction. (a) Predicted vs. actual output SST count, colored by level transition. (b) CDF of the relative output-size error.

그러나 이 실험으로도 다른 대안인 flush-only, fillseq보다 빠른 로딩 시간을 달성하지 못한다. 이는 BlobDB가 compaction의 비용을 감소시켰지만, flush-only와 fillseq는 그 비용 자체를 제거했기 때문이다. Section 3의 대안들은 intermediate SST를 생성하는 비용 자체를 우회하여 로딩 시간을 더욱 단축한다.

F2Load은 동일한 데이터셋에서 134.7초에 로딩을 마치므로, 두 write-path optimization보다 한 자릿수 이상 빠르다. 이는 F2Load이 compaction의 비용을 줄이는 대신 intermediate SST의 KV payload 자체를 기록하지 않기 때문이다.

## 5.3 Virtual Compaction Accuracy

Reproducing the final state correctly depends on how accurately virtual compaction predicts each compaction’s output. A wrong prediction produces a wrong DB state. To evaluate this, we ran a baseline RocksDB load. For every compaction, we built vSSTs from its input SSTs and ran virtual compaction on them in parallel. We collected 21,828 compaction jobs and evaluated the accuracy of its two tasks, merge and split, and deduplication.

**Merge and split** Virtual compaction predicts the exact output SST count for 90.9% of jobs and stays within  $\pm 1$  file for 99.8%, with a maximum error of  $\pm 3$  (Figure 10a). This holds across all level transitions, regardless of compaction depth. The output size is just as close (Figure 10b), with a median error of 1.15%.

**Deduplication** Deduplication relies on the KMV sketch’s cardinality estimate, which has a mean absolute error of 1.62% and a median of 1.16% (max 11.7%). This keeps both the deduplication correction and the size prediction accurate. Overall, virtual compaction reproduces the count and size of compaction output to within a few percent using only descriptors.

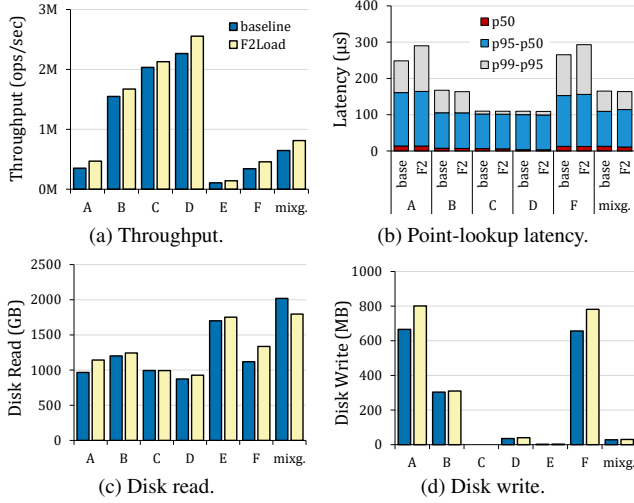


Figure 11: Workload behavior on the 10 TB dataset under YCSB (A–F) and MixGraph (mixg.). Each plot compares the baseline with the F2Load-built state. Latency in (b) is point-lookup latency at p50, p95, and p99, so the scan-based YCSB E is omitted. *base* and *F2* denote the baseline and F2Load.

## 5.4 Final State Fidelity

F2Load’s goal is to reproduce the state that incremental construction would build. F2Load predicts compaction outputs rather than performing them. We must therefore check that the reproduced state is close to the baseline. We verify this in two ways. The first is a large-scale case study on a single 10 TB dataset of 1 KB KV pairs. The second is a sweep across various configurations. We compare the two states by analyzing how the same benchmark behaves on them and their structural properties such as final DB size, SST count, and average SST size.

**Large-scale case study** We build a single 10 TB dataset of 1 KB KV pairs with no compression. On both the baseline state and the F2Load-built state, we run seven workloads (YCSB A–F [9] and MixGraph [6]) and compare throughput, tail latency, disk read, and disk write (Figure 11).

Disk read is the I/O that point lookups issue, and F2Load tracks the baseline on every workload. Disk write comes from compaction, so it appears only in workloads that write. The read-only workload C therefore produces almost no write, while the write-heavy workloads A and F trigger the most. Tail latency also tracks the baseline closely. These results indicate that the state built by F2Load preserves the baseline’s read and compaction behavior across the evaluated workloads.

**Configuration sweep** To confirm that this fidelity is not specific to one setting, we sweep 12 configurations that combine two KV sizes (1 KB, 91 B), two DB sizes (500 GB, 1 TB), and three key patterns (uniform and zipfian at 50% unique keys,

and unique at 100% unique keys). For each configuration, Figure 12 reports F2Load’s value normalized to the baseline for seven metrics in two groups. Three capture the loaded state (DB size, SST count, and SST size), and four capture how a read-only workload (YCSB C, zipfian 0.99) behaves on it (throughput, average latency, disk read, and SST access). An SST access adds no disk I/O while RocksDB pins each SST’s filter and index in memory, but it would bound read throughput under memory pressure. Across the configurations F2Load stays close to the baseline on every metric.

**Comparison against the design alternatives** We place F2Load on the same 1 TB dataset used for the design alternatives of Section 3 (1,000 GiB of 24 B keys and 1,000 B values) so that its state can be read against theirs. Loading takes 134.7 seconds in total: a 97.0-second run that includes materialization, which leaves 16.07 GiB of estimated pending compaction, followed by 37.8 seconds during which a checkpoint reopened under the common clean release drains that backlog to zero pending bytes. The completion stage reads 21.64 GiB and writes 19.55 GiB of real SSTs, and the reported time is the sum of the two process wall times, so it includes that additional physical compaction. All reads below run on this completed DB under the reader of Figure ??.

On filter checks per lookup, F2Load sits where the baseline sits and where no alternative does. It checks 3.53 filters per request with 18.29% of those checks positive, against the baseline’s 3.98 and 16.69%; Flush-only inflates the same metric to 10254.44 checks, while one-shot compaction and Fillseq collapse it to 1.00. Read throughput is  $1.04\text{--}1.70\times$  the baseline across the four cache configurations.

Live-key membership is where the current prototype falls short. The metadata-only path does not carry original key identity, so it reconstructs the key set approximately from a KMV sketch and a learned-index descriptor. The resulting successful-lookup fraction is 61.81% against the baseline’s 63.21%, a gap of 1.40 percentage points. F2Load therefore reproduces candidate-SST composition, which no alternative does, but not exact key membership; the throughput above should be read as evidence in that context rather than as a controlled layout-only speedup. Each cell is a single run, and F2Load was measured after the five alternative states rather than interleaved with them.

## 5.5 Memory Overhead

F2Load holds an in-memory descriptor for every SST, so its memory grows with the dataset. We measure this on the 91 B configuration, which packs the most records and stresses descriptor memory the most. The learned index dominates each descriptor. The metadata and KMV sketch take a fixed 16.6 KB, while the learned index adds one 32 B segment per run of keys within its error bound, about one segment per 20 distinct records or roughly 1.6 B per record. Resident memory

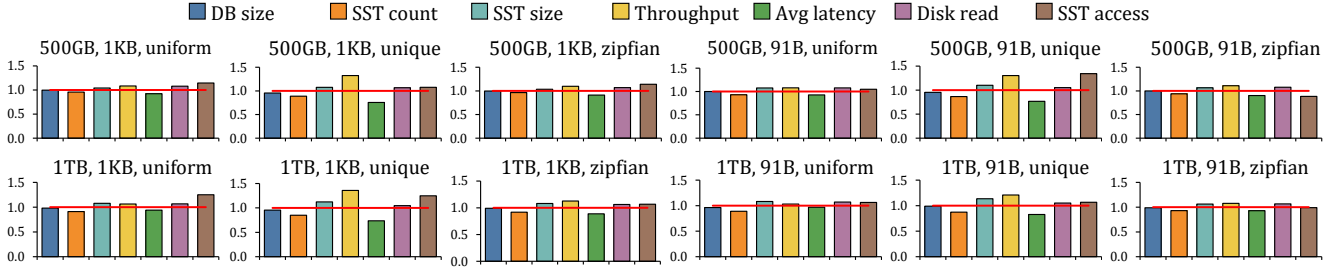


Figure 12: Configuration sweep across 12 settings. Each bar is F2Load’s value normalized to the baseline (1.0), and a value near 1.0 means F2Load matches the baseline.

therefore grows near-linearly across a  $16\times$  size range, from 7.0 GB at 500 GB to 91.2 GB at 8 TB, with the fixed KMV part under 2% of the total. This fits within DRAM, using under 10% of the server’s 1 TB at 8 TB, and is tunable, since relaxing the error bound trades index accuracy for less memory.

## 6 Conclusion

Loading a dataset is the first step of every LSM-Tree evaluation, and at terabyte scale it becomes a bottleneck on its own, because compaction rewrites the same data many times. We presented F2Load, which reproduces this final state without paying that cost. Its key idea is that the outcome of compaction can be predicted from a compact summary of each SST. F2Load therefore replaces each SST with an in-memory virtual SST, runs virtual compaction along the original compaction path, and materializes real SSTs only once the final state is fixed. It avoids intermediate materialization; the measured prototype also performs physical completion compactions after materialization. In the common 1,000 GiB, 1 KB comparison, F2Load reduces loading time by  $25.99\times$  and device writes by 94.1%, including physical completion compactions. The separate historical scaling series builds an 8 TB dataset of 91 B pairs in under two hours, a load the baseline would need a projected 47 hours to finish. Separate historical experiments compare the reproduced state and workload behavior across configurations. The new common comparison also exposes a key-membership difference, so its read-throughput ratios do not isolate layout effects.

## References

- [1] Apache HBase. Compression and data block encoding in HBase. <https://hbase.apache.org/docs/compression/>. Accessed: 2026-06-04.
- [2] Timothy G. Armstrong, Vamsi Ponnekanti, Dhruba Borthakur, and Mark Callaghan. LinkBench: A database benchmark based on the facebook social graph. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, pages 1185–1196. ACM, 2013.
- [3] Ziv Bar-Yossef, T. S. Jayram, Ravi Kumar, D. Sivakumar, and Luca Trevisan. Counting distinct elements in a data stream. In *Proceedings of the 6th International Workshop on Randomization and Approximation Techniques (RANDOM)*, pages 1–10. Springer, 2002.
- [4] Laurent Bindschaedler, Ashvin Goel, and Willy Zwaenepoel. Hailstorm: Disaggregated compute and storage for distributed LSM-based databases. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 20)*, pages 301–316. ACM, 2020.
- [5] Dhruba Borthakur, Jonathan Gray, Joydeep Sen Sarma, Kannan Muthukkaruppan, Nicolas Spiegelberg, Hairong Kuang, Karthik Ranganathan, Dmytro Molkov, Aravind Menon, Samuel Rash, Rodrigo Schmidt, and Amitanand Aiyer. Apache hadoop goes realtime at facebook. In *Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data (SIGMOD ’11)*, pages 1071–1080. ACM, 2011.
- [6] Zhichao Cao, Siying Dong, Sagar Vemuri, and David H. C. Du. Characterizing, modeling, and benchmarking RocksDB key-value workloads at facebook. In *18th USENIX Conference on File and Storage Technologies (FAST)*, pages 209–223. USENIX Association, 2020.
- [7] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E. Gruber. Bigtable: A distributed storage system for structured data. In *7th USENIX Symposium on Operating Systems Design and Implementation (OSDI 06)*, pages 205–218. USENIX Association, 2006.
- [8] Audrey Cheng, Xiao Shi, Aaron Kabcenell, Shilpa Lawande, Hamza Qadeer, Jason Chan, Harrison Tin, Ryan Zhao, Peter Bailis, Mahesh Balakrishnan, Nathan

- Bronson, Natacha Crooks, and Ion Stoica. TAOBench: An end-to-end benchmark for social networking workloads. *Proceedings of the VLDB Endowment*, 15(9):1965–1977, 2022.
- [9] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC)*, pages 143–154. ACM, 2010.
- [10] Yifan Dai, Yien Xu, Aishwarya Ganesan, Ramnathan Alagappan, Brian Kroth, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. From WiscKey to Bourbon: A learned index for log-structured merge trees. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 155–171. USENIX Association, 2020.
- [11] Siying Dong, Andrew Kryczka, Yanqin Jin, and Michael Stumm. RocksDB: Evolution of development priorities in a key-value store serving large-scale applications. *ACM Transactions on Storage*, 17(4):1–32, 2021.
- [12] Martin Grambow, Fabian Lehmann, and David Bermbach. Continuous benchmarking: Using system benchmarking in build pipelines. In *2019 IEEE International Conference on Cloud Engineering (IC2E)*, pages 241–246. IEEE, 2019.
- [13] Inhwi Hwang, Sangjin Lee, Sunggon Kim, Hyeonsang Eom, and Yongseok Son. Z-LFS: A zoned namespace-tailored log-structured file system for commodity small-zone ZNS SSDs. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*, pages 547–562, Boston, MA, July 2025. USENIX Association.
- [14] Henrik Ingo and David Daly. Automated system performance testing at MongoDB. In *Proceedings of the Workshop on Testing Database Systems (DBTest ’20)*. ACM, 2020.
- [15] Hera Koo, Sungho Moon, Sangeun Chae, Wook-Hee Kim, Jongmoo Choi, and Beomseok Nam. From learning to recycling: A log-structured learned-less index. *Proceedings of the ACM on Management of Data*, 4(1):1–26, 2026.
- [16] Avinash Lakshman and Prashant Malik. Cassandra: A decentralized structured storage system. In *Proceedings of the 3rd ACM SIGOPS International Workshop on Large Scale Distributed Systems and Middleware (LADIS ’09)*, pages 1–6. ACM, 2009.
- [17] Hayley LeBlanc, Nathan Taylor, James Bornholt, and Vijay Chidambaram. SquirrelFS: Using the rust compiler to check file-system crash consistency. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 387–404, Santa Clara, CA, July 2024. USENIX Association.
- [18] Yongkun Li, Chengjin Tian, Fan Guo, Cheng Li, and Yinlong Xu. ElasticBF: Elastic bloom filter with hotness awareness for boosting read performance in large Key-Value stores. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 739–752, Renton, WA, July 2019. USENIX Association.
- [19] Lanyue Lu, Thanumalayan Sankaranarayanan Pillai, Hariharan Gopalakrishnan, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. WiscKey: Separating keys from values in ssd-conscious storage. In *14th USENIX Conference on File and Storage Technologies (FAST)*, pages 133–148. USENIX Association, 2016.
- [20] Chen Luo and Michael J. Carey. Lsm-based storage techniques: a survey. *The VLDB Journal*, 29(1):393–418, 2020.
- [21] Yoshinori Matsunobu, Siying Dong, and Herman Lee. MyRocks: LSM-tree database storage engine serving facebook’s social graph. *Proceedings of the VLDB Endowment*, 13(12):3217–3230, 2020.
- [22] Meta Database Engineering Team. RocksDB: Leveled compaction. <https://github.com/facebook/rocksdb/wiki/Leveled-Compaction>. Accessed: 2026-06-10.
- [23] Meta Platforms, Inc. Reduce write amplification by aligning compaction output file boundaries. <https://rocksdb.org/blog/2022/10/31/align-compaction-output-file.html>, 2022. Accessed: 2026-06-11.
- [24] Meta Platforms, Inc. RocksDB wiki: Block cache. <https://github.com/facebook/rocksdb/wiki/Block-Cache>, 2024. Accessed: 2026-06-11.
- [25] Jaehong Min, Chenxingyu Zhao, Ming Liu, and Arvind Krishnamurthy. eZNS: An elastic zoned namespace for commodity ZNS SSDs. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 461–477. USENIX Association, 2023.
- [26] Thanh H. D. Nguyen, Bram Adams, Zhen Ming Jiang, Ahmed E. Hassan, Mohamed Nasser, and Parminder Flora. Automated detection of performance regressions using statistical process control techniques. In *Proceedings of the Third Joint WOSP/SIPEW International Conference on Performance Engineering (ICPE ’12)*, pages 299–310. ACM, 2012.
- [27] Patrick O’Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O’Neil. The log-structured merge-tree (lsm-tree). *Acta Informatica*, 33(4):351–385, 1996.

- [28] David Reinsel, John Gantz, and John Rydning. The digitization of the world – from edge to core. White Paper US44413318, International Data Corporation (IDC), 2018. Sponsored by Seagate.
- [29] RocksDB. Benchmarking tools. <https://github.com/facebook/rocksdb/wiki/Benchmarking-tools>. Accessed: 2026-06-04.
- [30] RocksDB. Compaction trivial move. <https://github.com/facebook/rocksdb/wiki/Compaction-Trivial-Move>. Accessed: 2026-06-04.
- [31] RocksDB. Compression. <https://github.com/facebook/rocksdb/wiki/Compression>. Accessed: 2026-06-04.
- [32] RocksDB. Integrated BlobDB. <https://rocksdb.org/blog/2021/05/26/integrated-blob-db.html>, 2021. Accessed: 2026-09-04.
- [33] Samsung Electronics. *Samsung V-NAND SSD PM9A3 Data Sheet*. Samsung Electronics, August 2022. Revision 1.0.
- [34] TiKV. TiKV overview. <https://tikv.org/docs/6.1/concepts/overview/>. Accessed: 2026-06-03.
- [35] Fenggang Wu, Ming-Hong Yang, Baoquan Zhang, and David H. C. Du. AC-Key: Adaptive caching for LSM-based key-value stores. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 603–615. USENIX Association, 2020.
- [36] YCSB. Yahoo! Cloud Serving Benchmark. <https://github.com/brianfrankcooper/YCSB>. Accessed: 2026-06-06.
- [37] Jinghuan Yu, Sam H. Noh, Young ri Choi, and Chun Jason Xue. ADOC: Automatically harmonizing dataflow between components in log-structured key-value stores for improved performance. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*, pages 65–80. USENIX Association, 2023.
- [38] Huanchen Zhang, Hyeontaek Lim, Viktor Leis, David G. Andersen, Michael Kaminsky, Kimberly Keeton, and Andrew Pavlo. SuRF: Practical range query filtering with fast succinct tries. In *Proceedings of the 2018 International Conference on Management of Data*, pages 323–336. ACM, 2018.